

Green Pace

Security Policy Presentation

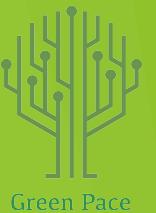
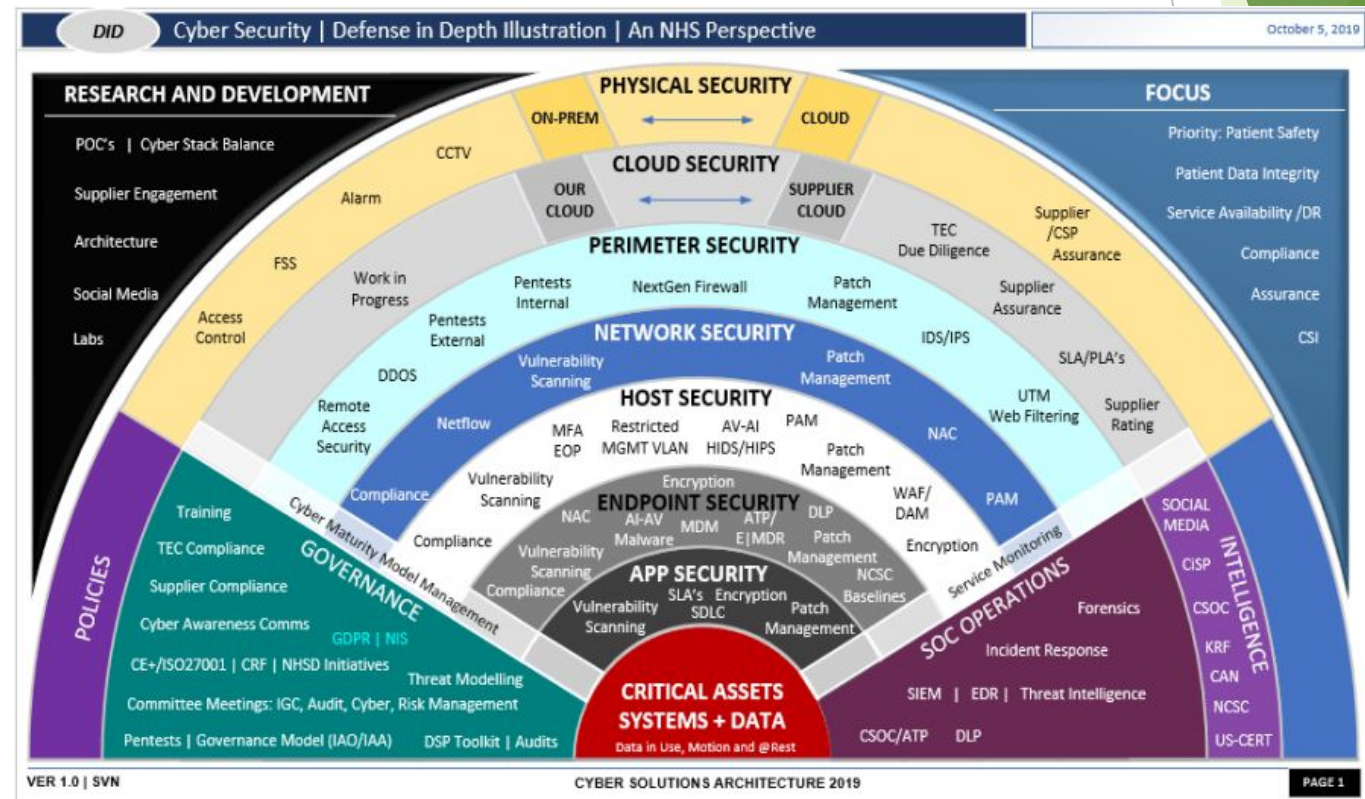
Developer: Miguel A Rodao



Green Pace

OVERVIEW: DEFENSE IN DEPTH

It is a cybersecurity approach that employs various overlapping security layers such as physical, technical, and administrative to safeguard assets. Based on the premise that no single defense is infallible, ensuring that if intruders penetrate one obstacle, additional ones are ready to manage the danger, minimizing risk and enhancing resilience against different assaults, from phishing to network intrusions.

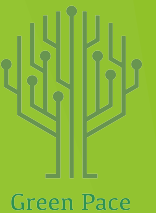


THREATS MATRIX

Rule	Severity	Likelihood	Remediation Cost	Priority	Level
STD-005-CPP	High	Likely	Medium	27 High	1
STD-003-CPP	High	Likely	Medium	18 High	1
STD-004-CPP	Medium	Probable	Medium	12 High	1
STD-007-CPP	High	Likely	Medium	9 Medium	2
STD-010-CPP	Low	Likely	Medium	9 Medium	2
STD-009-CPP	High	Probable	High	6 Medium	2
STD-006-CPP	Medium	Probable	High	4 Low	3
STD-001-CPP	Medium	Unlikely	Low	4 Low	3
STD-002-CPP	Medium	Unlikely	Medium	4 Low	3
STD-008-CPP	Medium	Unlikely	Low	4 Low	3

10 PRINCIPLES

1. Validate Input Data
2. Heed Compiler Warnings
3. Architect and Design for Security Policies
4. Keep it Simple
5. Default Deny
6. Adhere to the Principle of Least Privilege
7. Sanitize Data Sent to Other Systems
8. Practice Defense in Depth
9. Use Effective Quality Assurance Techniques
10. Adopt a Secure Coding Standard



CODING STANDARDS

- ▶ The order of the Coding Standards is based on priority, and it goes from High as the first one to Low for the last ones. They are color coded as High with red, Medium with yellow, and Low with green. They are further arranged on the same priority by likelihood, and it goes from the most likely to the least.

Rule	Severity	Likelihood	Priority
STD-005-CPP	High	Likely	27 High
STD-003-CPP	High	Likely	18 High
STD-004-CPP	Medium	Probable	12 High
STD-007-CPP	High	Likely	9 Medium
STD-010-CPP	Low	Likely	9 Medium
STD-009-CPP	High	Probable	6 Medium
STD-006-CPP	Medium	Probable	4 Low
STD-001-CPP	Medium	Unlikely	4 Low
STD-002-CPP	Medium	Unlikely	4 Low
STD-008-CPP	Medium	Unlikely	4 Low

ENCRYPTION POLICIES

Encryption at rest	Encryption at rest safeguards data that is stored. This can consist of cloud assets, phones, computers, and hard drives, among other items. Data protection can be achieved using encryption tools, disk encryption, and securing mobile devices and computers.
Encryption in flight	Encryption in transit focuses on safeguarding data that is on the move. This may occur between two devices within a network or when transferring outside of a network. This can be safeguarded through methods like email encryption, data loss prevention solutions, and robust network security measures, including firewalls and authentication. It is also crucial to evaluate the route data might be following, as well as the safety of this route.
Encryption in use	Encryption in use secures data that is generated, modified, or otherwise designated as actively in-use. This data can be safeguarded by guaranteeing that data control and protection are established before its use and are already implemented. Controlling access permissions and identity will further reduce risk to this data.

TRIPLE-A POLICIES

Authentication	Authentication refers to the process of verifying an individual's identity. This can encompass various forms, commonly including static passwords, one-time passwords, certificates, and biometric identifiers. These types of identification serve to confirm that an individual is genuinely who they say they are.
Authorization	Authorization defines the access rights and privileges granted to a user and is a crucial element of information and computer security. While authentication verifies an identity, authorization specifies what a user is allowed to access, thereby restricting potential vulnerabilities when individuals engage with sensitive information they shouldn't access, or the permissions granted during that access.
Accounting	Accounting involves monitoring activities when engaging with a system, displaying timestamps, utilized resources, and details on data transfers. This is useful for establishing a “bread crumb trail” in user behavior, as well as for forensic analysis and investigation, if necessary.

Unit Testing

```
// Create a test to verify exception when resizing to a negative
TEST_F(CollectionTest, ResizingToNegative) {
    // if throw exception for resising to a negative
    EXPECT_ANY_THROW(collection->resize(-1));
}
```

Unit Testing

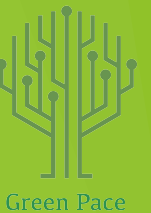
```
// Create 2 unit tests of your own to test something on the collection - do 1 positive & 1 negative
// Create a test to verify size increase of the vector when push_back a new value
TEST_F(CollectionTest, SizeIncreasePushBack) {
    // create collection
    add_entries(5);
    // add value to the vector
    collection->push_back(25);

    // if collection increase in size
    ASSERT_EQ(collection->size(), 6);
}
```

Unit Testing

```
// Create a test to verify the std::out_of_range exception is thrown when calling at() with an index out of bounds
// NOTE: This is a negative test
TEST_F(CollectionTest, OutOfRangeExcpCallAt) {
    // create collection
    add_entries(5);

    // if throw exception for std::out_of_range
    EXPECT_THROW(collection->at(10), std::out_of_range);
}
```



Unit Testing

```
// Create a test to verify reserve increases the capacity but not the size of the collection
TEST_F(CollectionTest, ReserveIncreaseCapacityNOTSize) {
    // create collection
    add_entries(5);

    // assign variables
    size_t oldCapacity = collection->capacity();
    size_t oldSize = collection->size();

    // reserve collection
    collection->reserve(10);

    // if size is equal then true
    ASSERT_TRUE(collection->size() == oldSize);
    // if capacity is bigger then true
    ASSERT_TRUE(collection->capacity() > oldCapacity);
}
```

Unit Testing

```
// Create a test to verify erase(begin,end) erases the collection
TEST_F(CollectionTest, EraseBeginToEnd) {
    // create collection
    add_entries(5);

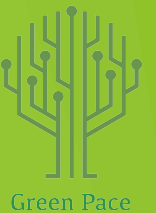
    // erase collection
    collection->erase(collection->begin(), collection->end());
    // if erase, the size must be nullptr
    ASSERT_NE(collection.get(), nullptr);
}
```

Unit Testing

```
// Create a test to verify resizing decreases the collection to zero
TEST_F(CollectionTest, ResizingDecreaseToZero) {
    // define default size
    size_t DEFAULT_SIZE = 5;
    // create collection with default size
    add_entries(DEFAULT_SIZE);

    // decrease the collection size
    collection->resize(0);

    // if empty, the size must be 0
    ASSERT_EQ(collection->size(), 0);
}
```



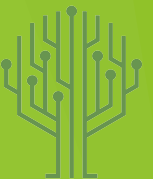
Unit Testing

```
Microsoft Visual Studio Debug Console
Running main() from D:\a_work\1\s\googletest\googletest\src\gtest_main.cc
[-----] Running 16 tests from 1 test case.
[-----] Global test environment set-up.
[-----] 16 tests from CollectionTest
[ RUN ] CollectionTest.CollectionSmartPointerIsNotNull
[ OK ] CollectionTest.CollectionSmartPointerIsNotNull (0 ms)
[ RUN ] CollectionTest.IsEmptyOnCreate
[ OK ] CollectionTest.IsEmptyOnCreate (0 ms)
[ RUN ] CollectionTest.AlwaysFail
[ FAILED ] CollectionTest.AlwaysFail (0 ms)
E:\SNHU\CS405\GoogleTest\GoogleTest\test.cpp(87): error: Failed
[ RUN ] CollectionTest.CanAddToEmptyVector
[ OK ] CollectionTest.CanAddToEmptyVector (0 ms)
[ RUN ] CollectionTest.CanAddFiveValuesToVector
[ OK ] CollectionTest.CanAddFiveValuesToVector (0 ms)
[ RUN ] CollectionTest.MaxSizeGreaterEqual
[ OK ] CollectionTest.MaxSizeGreaterEqual (0 ms)
[ RUN ] CollectionTest.CapacityGreaterEqual
[ OK ] CollectionTest.CapacityGreaterEqual (0 ms)
[ RUN ] CollectionTest.ResizingIncrease
[ OK ] CollectionTest.ResizingIncrease (0 ms)
[ RUN ] CollectionTest.ResizingDecrease
[ OK ] CollectionTest.ResizingDecrease (0 ms)
[ RUN ] CollectionTest.ResizingDecreaseToZero
[ OK ] CollectionTest.ResizingDecreaseToZero (0 ms)
[ RUN ] CollectionTest.ClearErase
[ OK ] CollectionTest.ClearErase (0 ms)
[ RUN ] CollectionTest.EraseBeginToEnd
[ OK ] CollectionTest.EraseBeginToEnd (0 ms)
[ RUN ] CollectionTest.ReserveIncreaseCapacityNOTSize
[ OK ] CollectionTest.ReserveIncreaseCapacityNOTSize (0 ms)
[ RUN ] CollectionTest.OutOfRangeExcpCallAt
[ OK ] CollectionTest.OutOfRangeExcpCallAt (0 ms)
[ RUN ] CollectionTest.SizeIncreasePushBack
[ OK ] CollectionTest.SizeIncreasePushBack (0 ms)
[ RUN ] CollectionTest.ResizingToNegative
[ OK ] CollectionTest.ResizingToNegative (0 ms)
[-----] 16 tests from CollectionTest (7 ms total)

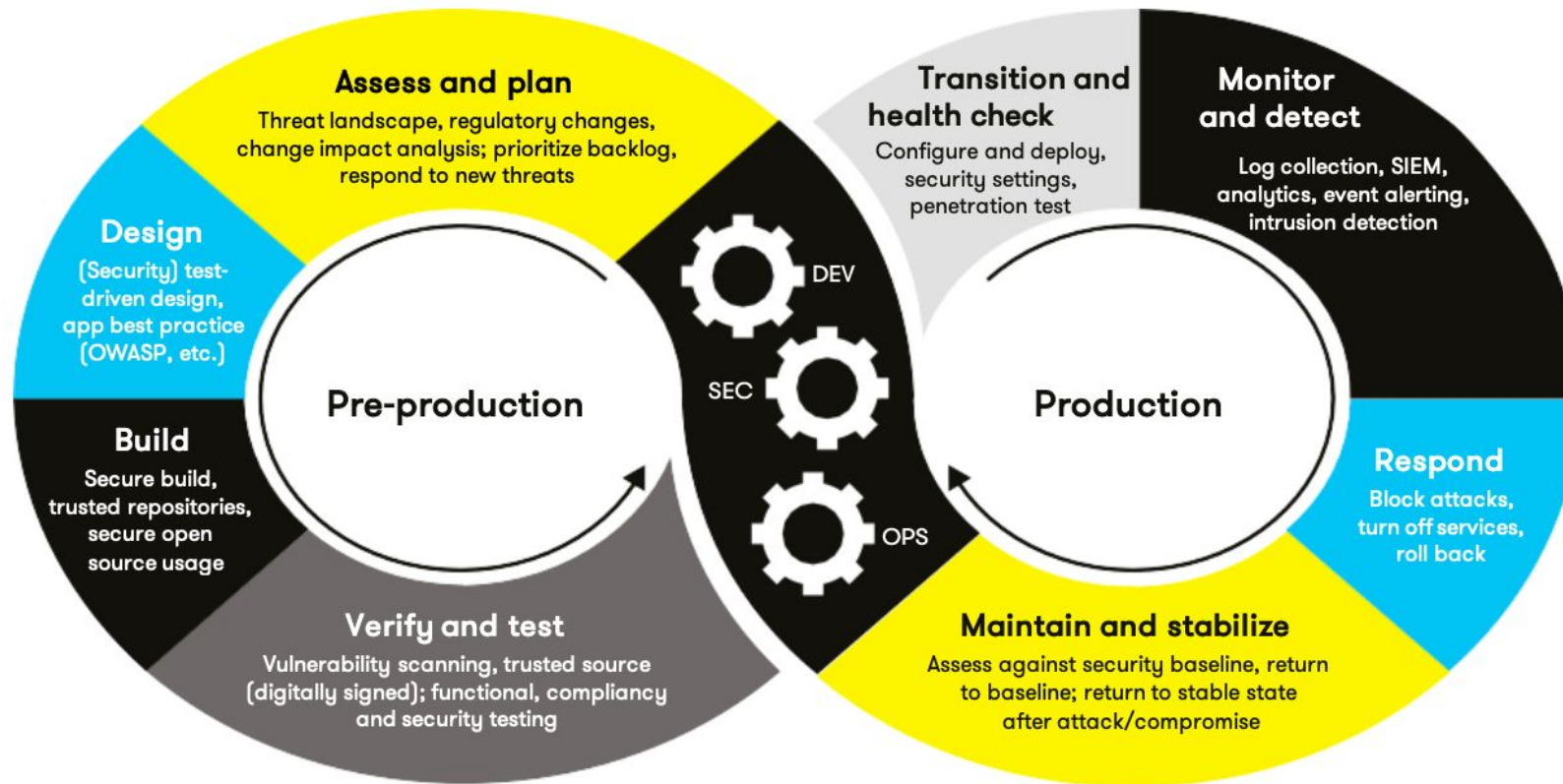
[-----] Global test environment tear-down
[-----] 16 tests from 1 test case ran. (8 ms total)
[ PASSED ] 15 tests.
[ FAILED ] 1 test, listed below:
[ FAILED ] CollectionTest.AlwaysFail

1 FAILED TEST

E:\SNHU\CS405\GoogleTest\x64\Debug\GoogleTest.exe (process 24964) exited with code 1 (0x1).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```



AUTOMATION SUMMARY



TOOLS

DevSecOps is an automated system that incorporates security at each phase of the Continuous Integration/Continuous Deployment process or SDLC Software Development Life Cycle. Identifies and corrects vulnerabilities early, resulting in quicker, safer, and compliant software releases.

Tools:

Parasoft C/C++ Test: Employed in the design phase, verification, and testing stage during pre-production.

CppCheck: Used for its primary purpose - to detect bugs and potential issues in source code that compilers might not catch.

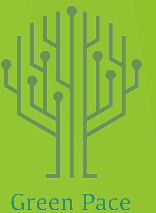
Logentries: Utilized in the monitoring and detection phase of post-production for automating log information.

RISKS AND BENEFITS

1. Hackers can access sensitive customer data:

How it happens: When DevSecOps is ignored, weak points remain hidden. Attackers can exploit these vulnerabilities before anyone warns.

Example: In 2019, Capital One's misconfigured firewall exposed data of 100 million customers. This is a clear case of DevSecOps failure.

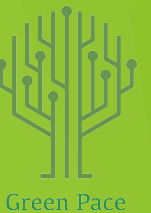


RISKS AND BENEFITS

2. Fixing problems later costs more than avoiding them early:

How it happens: If DevSecOps is ignored, security problems are discovered too late. This results in high maintenance, legal fees, and settlements.

Example: In 2017, Equifax paid over \$700 million after failing to fix a known security issue, showing the high cost of ignoring DevSecOps.

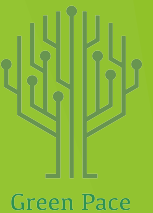


RISKS AND BENEFITS

3. Customers lose trust in the company:

How it happens: Ignoring DevSecOps can raise doubts about whether their data is safe.

Example: In 2013, Target's breach exposed 40 million credit card numbers, resulting in a valuation \$18.5 million in settlements and long-term trust.

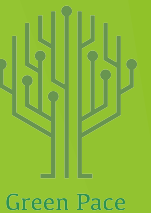


RISKS AND BENEFITS

4. Important systems can stop functioning, leading to reduced productivity:

How it happens: Digital attacks can reduce efficiency when DevSecOps in cloud environments .

Example: Fuel shortages spread across the US East Coast after cyber threats disrupted the Colonial Pipeline.



RECOMMENDATIONS

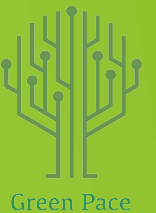
This policy provides a strong basis. Needs to undergo periodic reviews and revisions, or whenever gaps are detected.

Yearly evaluations from an external source, like a white hat security company, will assist in providing a practical assessment of security and identify potential weaknesses.

Enforcement guidelines for every stage - The policy lacks a clear definition regarding enforcement beyond automated tools. In what ways, locations, and areas of expertise will be utilized to guarantee that policies are adhered to appropriately and effectively

CONCLUSIONS

Embracing optimal practices and coding standards, implementing defense in depth, continually evaluating the motives of possible attacks, and maintaining a “no one is safe” mindset regarding security and prevention during development will aid in evolving DevOps into DevSecOps at Green Pace.



REFERENCES

wal, L. (Ed.). (2025, September 29). *Real-world Consequences of Ignoring DevSecOps In The Business App*. IFour Blog | Custom Software Development USA | Offshore Software Development - Ifourtechnolab.com; iFour Technolab Pvt. Ltd.
<https://www.ifourtechnolab.com/blog/impact-of-ignoring-devsecops>